

Technical Documentation

MAFGA Multi-Agent POC

Version: 1.0.0

Date: April 2026

Architecture Pattern: MAF 1.0 GA (Microsoft Agent Framework)

Table of Contents

1. System Overview
2. Technology Stack
3. Module Architecture
4. Data Flow
5. Agent Implementation
6. Orchestration Engine
7. Protocol Implementations
8. Security Implementation
9. State Management
10. Observability Stack
11. Deployment Architecture
12. Performance Considerations
13. Error Handling
14. Testing Strategy

1. System Overview

1.1 Purpose

The MAFGA Multi-Agent POC implements a production-ready multi-agent system following the **Microsoft Agent Framework 1.0 GA** architecture. It demonstrates:

- Cloud-agnostic LLM integration
- Multi-agent orchestration patterns
- Human-in-the-loop workflows
- Standard protocol compliance (MCP, A2A)

1.2 Design Principles

Principle	Implementation
Modularity	Each component is independently deployable
Cloud Agnostic	Factory pattern abstracts LLM providers
Security First	EntraID auth, RBAC, content safety
Observable	Full OpenTelemetry instrumentation
Testable	Dependency injection, mock-friendly

1.3 Key Architectural Decisions

Decision	Rationale
Python 3.11 +	Async support, type hints, pattern matching
FastAPI	High performance, OpenAPI auto-generation
Pydantic v2	Runtime validation, settings management
Redis	Distributed memory, pub/sub for HITL
OpenTelemetry	Vendor-neutral observability

2. Technology Stack

2.1 Core Dependencies

```
# Runtime
python >= 3.11
fastapi >= 0.109.0
uvicorn >= 0.27.0
pydantic >= 2.6.0
pydantic-settings >= 2.1.0

# LLM Providers
openai >= 1.12.0
anthropic >= 0.18.0
google-generativeai >= 0.4.0

# Azure
azure-identity >= 1.15.0
azure-ai-projects >= 1.0.0b1
msal >= 1.26.0

# Data & Caching
redis >= 5.0.0
httpx >= 0.26.0

# Observability
opentelemetry-api >= 1.22.0
opentelemetry-sdk >= 1.22.0
opentelemetry-exporter-otlp >= 1.22.0
azure-monitor-opentelemetry-exporter >= 1.0.0b21

# Testing
pytest >= 8.0.0
pytest-asyncio >= 0.23.0
pytest-cov >= 4.1.0
```

2.2 Development Tools

```
# Code Quality
ruff >= 0.2.0
```

```
mypy >= 1.8.0
black >= 24.1.0

# Documentation
mkdocs >= 1.5.0
mkdocs-material >= 9.5.0
```

3. Module Architecture

3.1 Directory Structure

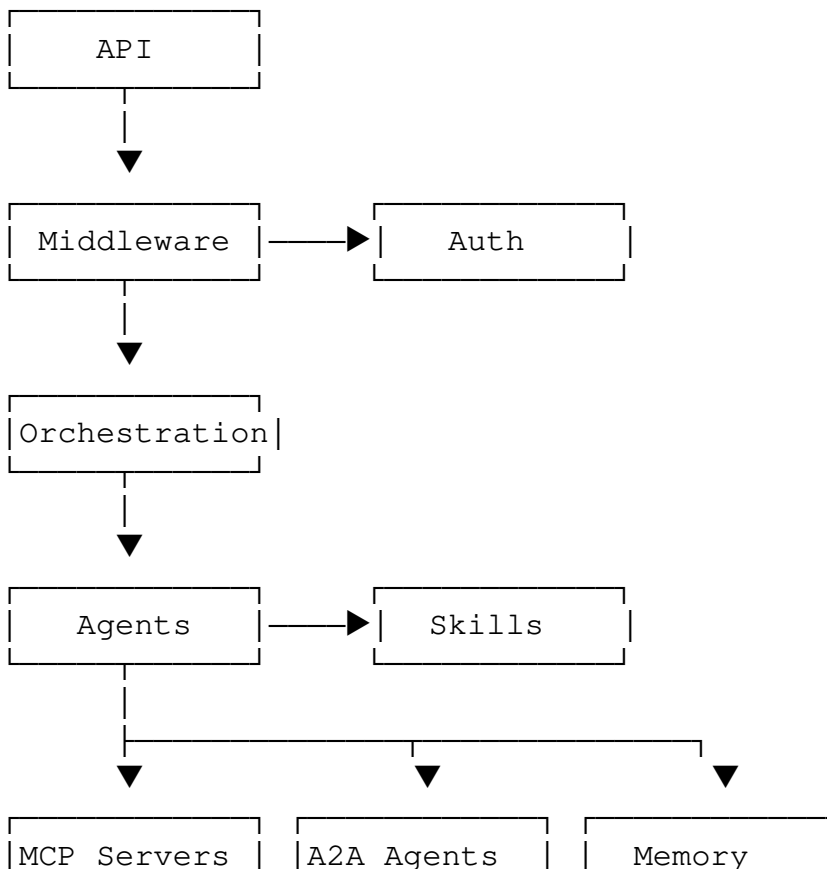
```
Multiagent-MAFGA-Arch/
├── api/                                # HTTP API Layer
│   ├── __init__.py
│   ├── main.py                        # FastAPI app factory
│   ├── routes.py                     # Route definitions
│   └── models.py                     # Pydantic schemas
├── agents/                            # Agent Implementations
│   ├── __init__.py
│   ├── agent_factory.py              # Cloud-agnostic factory
│   ├── base_agent.py                 # Abstract base class
│   ├── merch_planner.py
│   ├── space_planner.py
│   ├── loyalty_agent.py
│   ├── products_finder.py
│   ├── commercial_sales.py
│   └── campaign_analyst.py
├── orchestration/                     # Orchestration Patterns
│   ├── __init__.py
│   ├── magentic_orchestrator.py
│   ├── traffic_router.py
│   ├── parallel_executor.py
│   └── human_approval.py
├── mcp_servers/                       # MCP Tool Servers
│   ├── __init__.py
│   ├── base_mcp_server.py
│   ├── snowflake_mcp.py
│   ├── salesforce_mcp.py
│   ├── weather_mcp.py
│   ├── items_api_mcp.py
│   ├── personalisation_mcp.py
│   └── localisation_mcp.py
├── a2a_agents/                        # A2A Protocol
│   ├── __init__.py
│   ├── a2a_client.py
│   ├── a2a_server.py
│   └── mock_a2a_agents.py
├── middleware/                        # Request Pipeline
│   ├── __init__.py
│   ├── base_middleware.py
│   └── rbac_middleware.py
```

```

├── content_safetymiddleware.py
├── audit_logmiddleware.py
├── memory/                                # State Management
│   ├── __init__.py
│   ├── memory_provider.py
│   ├── redis_memory.py
│   └── in_memory.py
├── checkpoint/                          # Checkpointing
│   ├── __init__.py
│   ├── checkpoint_store.py
│   └── file_checkpoint.py
├── skills/                              # Skill System
│   ├── __init__.py
│   ├── skill_loader.py
│   ├── skill_registry.py
│   └── base_skill.py
├── observability/                       # Telemetry
│   ├── __init__.py
│   ├── telemetry.py
│   ├── tracing.py
│   └── metrics.py
├── auth/                                # Authentication
│   ├── __init__.py
│   └── entra_auth.py
├── config/                              # Configuration
│   ├── __init__.py
│   └── settings.py

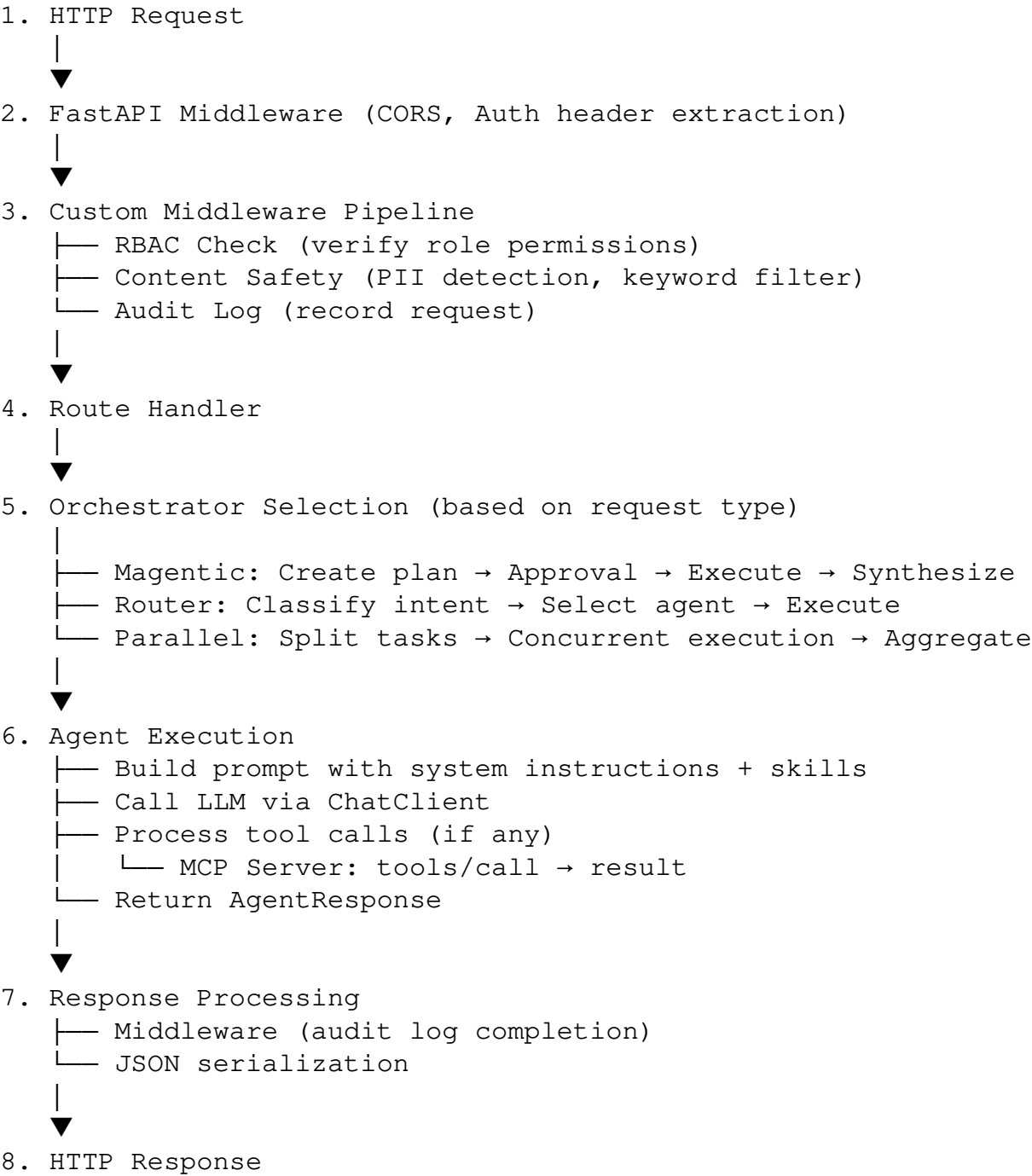
```

3.2 Module Dependencies



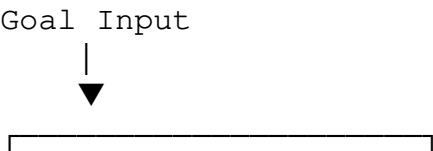
4. Data Flow

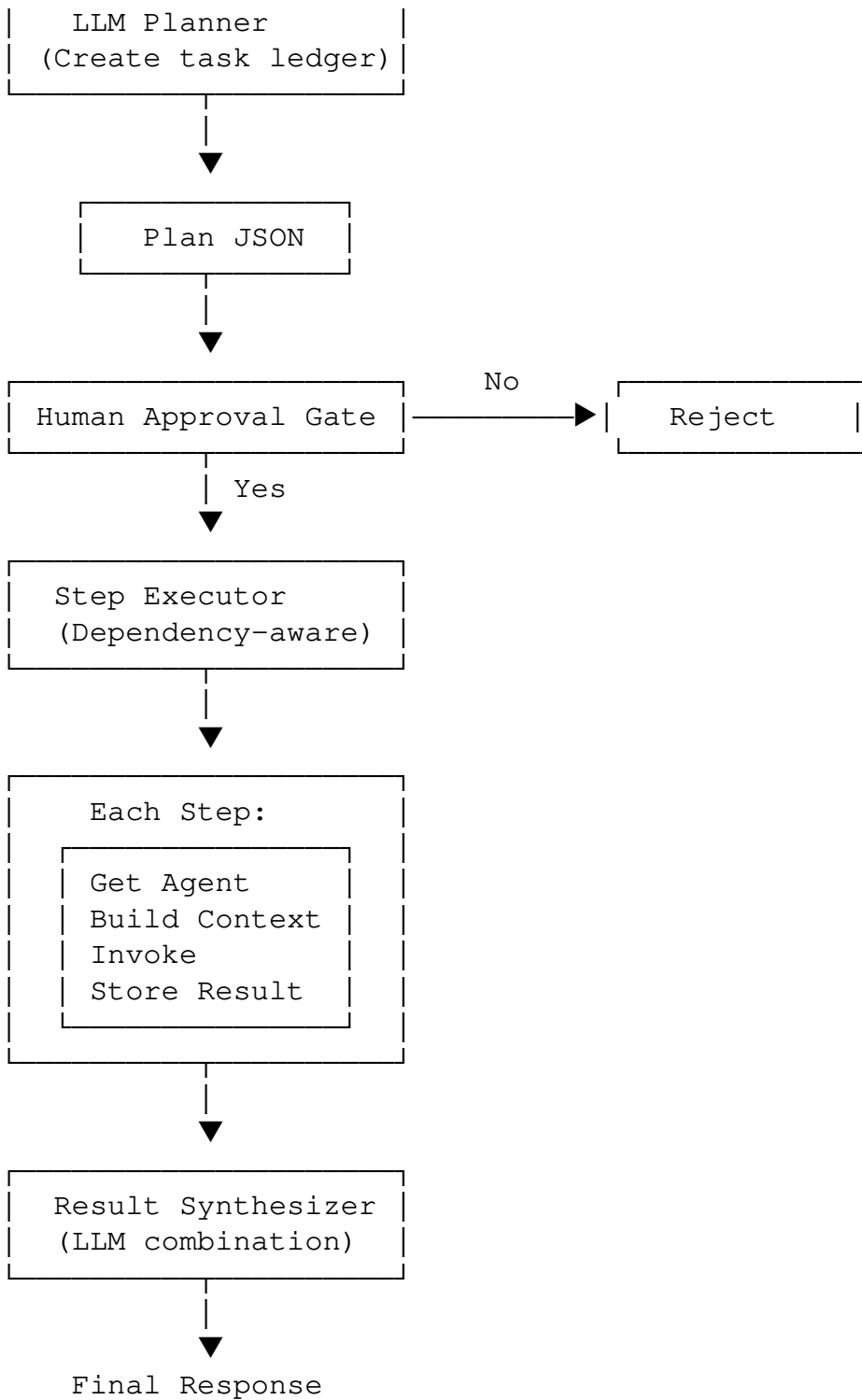
4.1 Request Flow



4.2 Orchestration Data Flow

Magentic Pattern





5. Agent Implementation

5.1 Agent Base Class

```
class BaseRetailAgent(ABC):  
    """  
    Abstract base agent implementing MAF 1.0 GA pattern.  
  
    Key Components:  
    - ChatClient: Cloud-agnostic LLM interface  
    - MCP Servers: Tool providers
```

- Skills: Domain knowledge injection

"""

```
def __init__(
    self,
    name: str,
    description: str,
    client: ChatClient,
    mcp_servers: list[BaseMCPServer] = None,
    skills: list[Skill] = None,
```

```
):
    self._name = name
    self._description = description
    self._client = client
    self._mcp_servers = mcp_servers or []
    self._skills = skills or []
```

@abstractmethod

```
def get_system_prompt(self) -> str:
    """Return agent-specific system instructions."""
    pass
```

```
async def invoke(self, query: str, context: dict = None) -> AgentRe
    """
```

Main entry point for agent invocation.

Flow:

1. Build messages array with system prompt + skills
2. Call ChatClient.chat_completion()
3. Process any tool calls via MCP
4. Return structured response

"""

```
# Build system prompt with skill injections
system_prompt = self.get_system_prompt()
for skill in self._skills:
    system_prompt += skill.get_prompt_injection()
```

```
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": query},
]
```

```
if context:
    messages.insert(1, {
        "role": "system",
        "content": f"Context: {json.dumps(context)}"
    })
```

```
# Get available tools
tools = self.get_tools()
```

```
# Call LLM
response = await self._client.chat_completion(
```

```

        messages=messages,
        tools=tools if tools else None,
    )

    # Process tool calls
    tool_results = []
    if response.get("tool_calls"):
        tool_results = await self._execute_tools(response["tool_cal

    return AgentResponse(
        content=response.get("content", ""),
        agent_name=self._name,
        tool_calls=tool_results,
        metadata={"model": response.get("model")},
    )

```

5.2 ChatClient Interface

```

class ChatClient(Protocol):
    """Cloud-agnostic LLM interface."""

    async def chat_completion(
        self,
        messages: list[dict],
        tools: list[dict] = None,
        temperature: float = 0.7,
        max_tokens: int = None,
    ) -> dict:
        """
        Returns:
        {
            "content": str,
            "tool_calls": list[dict] | None,
            "model": str,
            "usage": {"input_tokens": int, "output_tokens": int}
        }
        """
        ...

```

5.3 Factory Pattern

```

class AgentFactory:
    """Creates cloud-agnostic chat clients."""

    PROVIDERS = {
        "azure_foundry": "_create_azure_foundry_client",
        "azure_openai": "_create_azure_openai_client",
        "openai": "_create_openai_client",
        "anthropic": "_create_anthropic_client",
        "gemini": "_create_gemini_client",
        "ollama": "_create_ollama_client",
        "bedrock": "_create_bedrock_client",
    }

```



```

}

async def create_chat_client(
    self,
    provider: str = None,
) -> ChatClient:
    provider = provider or settings.agent_provider

    if provider not in self.PROVIDERS:
        raise ValueError(f"Unknown provider: {provider}")

    factory_method = getattr(self, self.PROVIDERS[provider])
    return await factory_method()

```

6. Orchestration Engine

6.1 Magentic Orchestrator

The Magentic pattern implements task-ledger based planning:

```

class MagenticOrchestrator:
    """
    MAF 1.0 GA Magentic Orchestration Pattern.

    Algorithm:
    1. PLAN: Use LLM to decompose goal into steps
    2. APPROVE: Optional human approval gate
    3. EXECUTE: Run steps respecting dependencies
    4. SYNTHESIZE: Combine results
    """

    async def run(self, goal: str, require_approval: bool = True) -> dict:
        # Phase 1: Planning
        plan = await self._create_plan(goal)

        # Phase 2: Approval
        if require_approval and self._approval_callback:
            approved = self._approval_callback(plan)
            if not approved:
                return {"status": "rejected", "plan": plan.to_dict()}

        # Phase 3: Execution
        results = await self._execute_plan(plan)

        # Phase 4: Synthesis
        synthesis = await self._synthesize_results(plan, results)

        return {
            "status": "completed",
            "result": synthesis,
            "plan": plan.to_dict(),
        }

```

```
        "agent_responses": results,
    }
```

6.2 Plan Structure

```
@dataclass
class MagenticPlan:
    plan_id: str
    goal: str
    steps: list[PlanStep]
    created_at: datetime
    approved: bool = False

    def get_ready_steps(self) -> list[PlanStep]:
        """Get steps with all dependencies satisfied."""
        completed_ids = {
            s.step_id for s in self.steps
            if s.status == PlanStepStatus.COMPLETED
        }

        return [
            s for s in self.steps
            if s.status == PlanStepStatus.PENDING
            and all(dep in completed_ids for dep in s.dependencies)
        ]

@dataclass
class PlanStep:
    step_id: str
    agent_name: str
    task: str
    dependencies: list[str]
    status: PlanStepStatus
    result: str = None
```

6.3 Dependency-Aware Execution

```
async def _execute_plan(self, plan: MagenticPlan) -> dict:
    """Execute steps respecting dependency DAG."""
    results = {}
    rounds = 0

    while not plan.is_complete() and rounds < self._max_rounds:
        rounds += 1
        ready_steps = plan.get_ready_steps()

        for step in ready_steps:
            step.status = PlanStepStatus.IN_PROGRESS

            # Build context from dependencies
            dep_context = self._build_dependency_context(plan, step)
```

```

        # Invoke agent
        agent = self._agents[step.agent_name]
        response = await agent.invoke(step.task, context=dep_context)

        step.result = response.content
        step.status = PlanStepStatus.COMPLETED
        results[step.step_id] = response

    return results

```

7. Protocol Implementations

7.1 MCP (Model Context Protocol)

Tool Definition Format

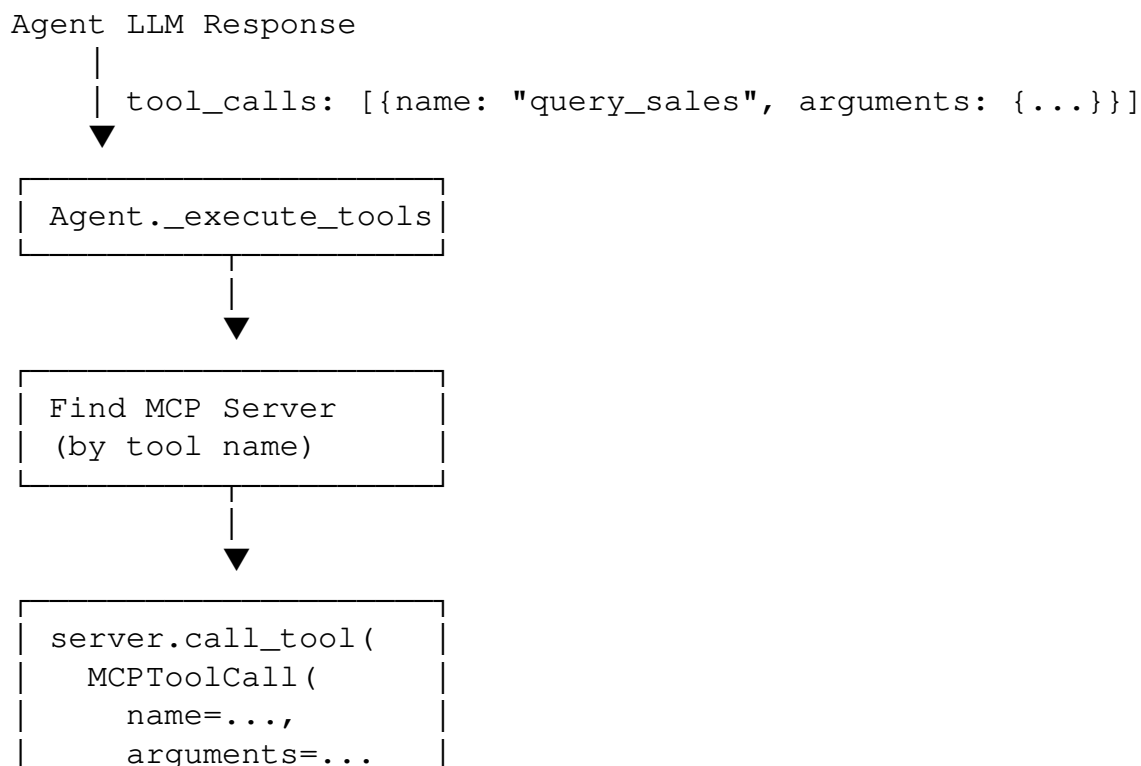
```

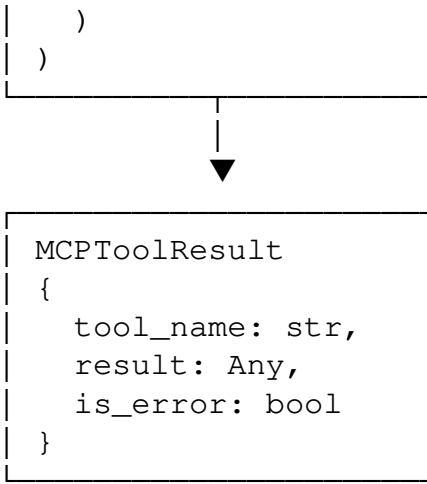
@dataclass
class MCPTool:
    name: str
    description: str
    input_schema: dict # JSON Schema

    def to_dict(self) -> dict:
        return {
            "name": self.name,
            "description": self.description,
            "inputSchema": self.input_schema,
        }

```

Tool Call Flow





7.2 A2A (Agent-to-Agent) Protocol

Message Format

```
@dataclass
class A2AMessage:
    message_id: str
    conversation_id: str
    sender_agent: str
    recipient_agent: str
    message_type: A2AMessageType # REQUEST | RESPONSE | HANDOFF
    content: str
    context: dict = None
    metadata: dict = field(default_factory=dict)
    created_at: datetime = field(default_factory=lambda: datetime.now(U
```

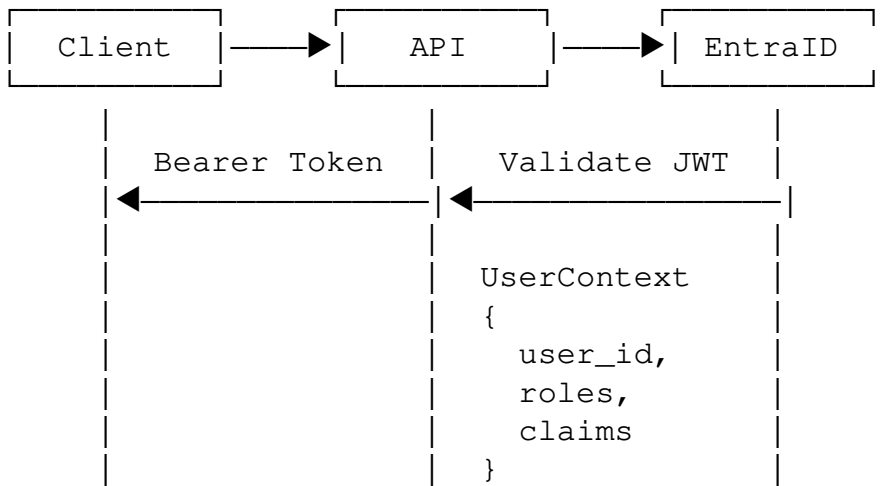
HTTP Transport

```
POST /a2a
Content-Type: application/json
X-A2A-Sender: LocalAgent

{
  "message_id": "msg-abc123",
  "conversation_id": "conv-xyz789",
  "sender_agent": "LocalAgent",
  "recipient_agent": "PricingAgent",
  "message_type": "request",
  "content": "What is the price for SKU-123?",
  "context": {"customer_tier": "gold"}
}
```

8. Security Implementation

8.1 Authentication Flow



8.2 RBAC Implementation

```

class RBACMiddleware(MiddlewareBase):
    """Role-based access control."""

    AGENT_ROLES = {
        "MerchPlanner": ["analyst", "planner", "manager", "admin"],
        "SpacePlanner": ["planner", "manager", "admin"],
        "CommercialSales": ["sales", "manager", "admin"],
        "LoyaltyAgent": ["marketing", "analyst", "admin"],
        "ProductsFinder": ["*"], # All roles
        "CampaignAnalyst": ["marketing", "analyst", "admin"],
    }

    async def process(self, request, context, next_middleware):
        user = context.get("user")
        agent_name = request.get("agent_name")

        if not user:
            return MiddlewareResult(blocked=True, reason="Unauthenticated")

        allowed_roles = self.AGENT_ROLES.get(agent_name, [])

        if "*" in allowed_roles:
            return await next_middleware(request, context)

        if not any(role in allowed_roles for role in user.roles):
            return MiddlewareResult(
                blocked=True,
                reason=f"Access denied: requires {allowed_roles}"
            )

        return await next_middleware(request, context)
  
```

8.3 Content Safety

```

class ContentSafetyMiddleware(MiddlewareBase):
    """Content filtering and PII detection."""
  
```

```

PII_PATTERNS = [
    (r'\b[\w.-]+@[ \w.-]+\.\w+\b', "email"),
    (r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b', "phone"),
    (r'\b\d{3}[-]?\d{2}[-]?\d{4}\b', "ssn"),
    (r'\b\d{16}\b', "credit_card"),
]

BLOCKED_KEYWORDS = ["password", "secret", "api_key"]

async def process(self, request, context, next_middleware):
    content = request.get("query", "")

    # Check for PII
    for pattern, pii_type in self.PII_PATTERNS:
        if re.search(pattern, content, re.IGNORECASE):
            return MiddlewareResult(
                blocked=True,
                reason=f"PII detected: {pii_type}"
            )

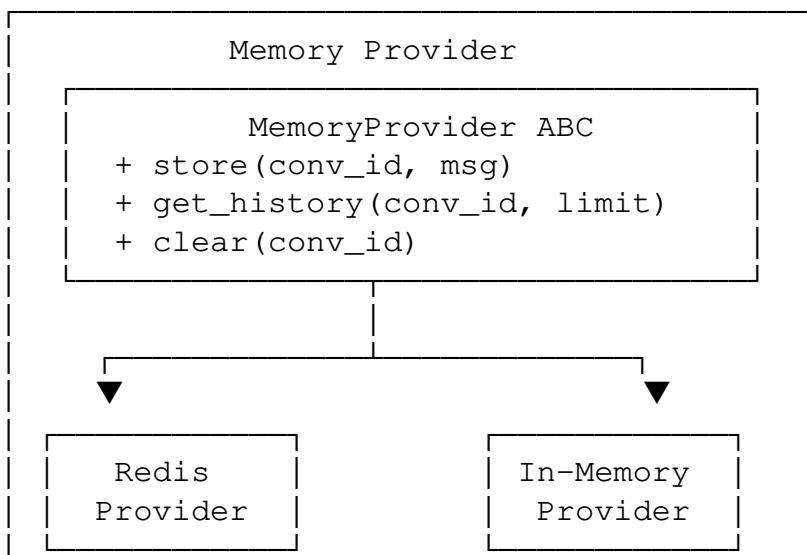
    # Check blocked keywords
    content_lower = content.lower()
    for keyword in self.BLOCKED_KEYWORDS:
        if keyword in content_lower:
            return MiddlewareResult(
                blocked=True,
                reason=f"Blocked keyword: {keyword}"
            )

    return await next_middleware(request, context)

```

9. State Management

9.1 Memory Architecture



9.2 Redis Implementation

```
class RedisMemoryProvider(MemoryProvider):
    """Redis-backed conversation memory."""

    def __init__(self, redis_url: str):
        self._redis = redis.from_url(redis_url)
        self._ttl = 3600 * 24  # 24 hour TTL

    async def store(self, conversation_id: str, message: dict) -> None:
        key = f"conv:{conversation_id}"
        timestamp = time.time()

        # Use sorted set for ordering
        await self._redis.zadd(
            key,
            {json.dumps(message): timestamp}
        )
        await self._redis.expire(key, self._ttl)

    async def get_history(
        self,
        conversation_id: str,
        limit: int = 50
    ) -> list[dict]:
        key = f"conv:{conversation_id}"

        # Get recent messages ordered by timestamp
        messages = await self._redis.zrange(
            key, -limit, -1
        )

        return [json.loads(m) for m in messages]
```

9.3 Checkpoint Store

```
class FileCheckpointStore(CheckpointStore):
    """File-based checkpoint persistence."""

    async def save(self, checkpoint_id: str, data: CheckpointData) -> None:
        path = self._checkpoint_dir / f"{checkpoint_id}.json"

        content = {
            "checkpoint_id": checkpoint_id,
            "data": data.to_dict(),
            "saved_at": datetime.now(UTC).isoformat(),
        }

        async with aiofiles.open(path, 'w') as f:
            await f.write(json.dumps(content, indent=2))
```

```

async def load(self, checkpoint_id: str) -> CheckpointData:
    path = self._checkpoint_dir / f"{checkpoint_id}.json"

    async with aiofiles.open(path, 'r') as f:
        content = json.loads(await f.read())

    return CheckpointData.from_dict(content["data"])

```

10. Observability Stack

10.1 OpenTelemetry Setup

```

def init_telemetry(config: TelemetryConfig) -> TelemetryProvider:
    """Initialize OpenTelemetry with configured exporters."""

    # Create resource
    resource = Resource.create({
        SERVICE_NAME: config.service_name,
        "service.version": "1.0.0",
        "deployment.environment": config.environment,
    })

    # Tracing
    tracer_provider = TracerProvider(resource=resource)

    if config.exporter_type == ExporterType.AZURE_MONITOR:
        exporter = AzureMonitorTraceExporter(
            connection_string=config.azure_connection_string
        )
    elif config.exporter_type == ExporterType.OTLP:
        exporter = OTLPSpanExporter(endpoint=config.otlp_endpoint)
    else:
        exporter = ConsoleSpanExporter()

    tracer_provider.add_span_processor(BatchSpanProcessor(exporter))
    trace.set_tracer_provider(tracer_provider)

    # Metrics
    # ... similar setup

    return TelemetryProvider(tracer_provider, meter_provider)

```

10.2 Tracing Decorators

```

def trace_agent_call(agent_name: str = None) -> Callable:
    """Decorator for tracing agent invocations."""

    def decorator(func: Callable) -> Callable:
        @functools.wraps(func)
        async def wrapper(*args, **kwargs):

```



```

tracer = get_tracer("agent")

with tracer.start_as_current_span(f"agent.{agent_name}") as
    span.set_attribute("agent.name", agent_name)
    span.set_attribute("agent.operation", "invoke")

    try:
        result = await func(*args, **kwargs)
        span.set_attribute("agent.success", True)
        return result
    except Exception as e:
        span.set_attribute("agent.success", False)
        span.record_exception(e)
        raise

    return wrapper
return decorator

```

10.3 Metrics Collection

```

class AgentMetrics:
    """Metrics collection for agents."""

    def __init__(self):
        meter = get_meter("agent_metrics")

        self._request_counter = meter.create_counter(
            "agent.requests",
            description="Total agent requests",
        )

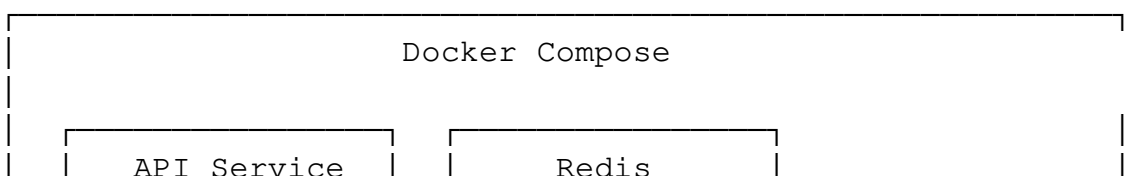
        self._latency_histogram = meter.create_histogram(
            "agent.latency",
            description="Request latency in ms",
            unit="ms",
        )

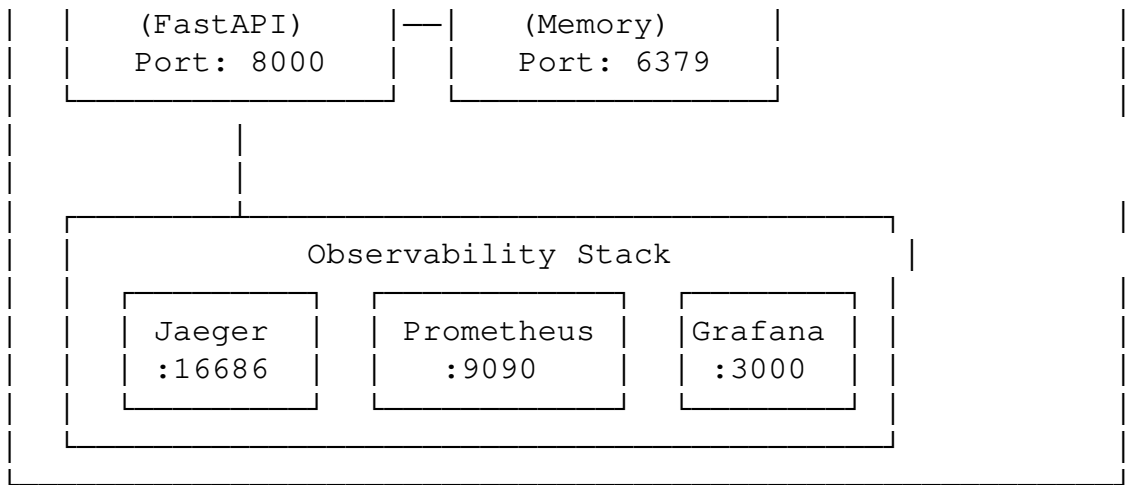
        self._error_counter = meter.create_counter(
            "agent.errors",
            description="Agent errors",
        )

```

11. Deployment Architecture

11.1 Container Architecture





11.2 Kubernetes Deployment (Production)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mafga-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mafga-api
  template:
    spec:
      containers:
      - name: api
        image: mafga-api:1.0.0
        ports:
        - containerPort: 8000
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "1000m"
        livenessProbe:
          httpGet:
            path: /live
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 5
```

12. Performance Considerations

12.1 Async Architecture

All I/O operations are async:

- LLM API calls
- Redis operations
- File I/O
- HTTP requests (A2A)

12.2 Connection Pooling

```
# Redis connection pool
redis_pool = redis.ConnectionPool(
    host=settings.redis_host,
    port=settings.redis_port,
    max_connections=20,
)

# HTTP client pool
http_client = httpx.AsyncClient(
    limits=httpx.Limits(max_connections=100),
    timeout=httpx.Timeout(30.0),
)
```

12.3 Caching Strategy

Data	Cache Location	TTL
Agent responses	Memory	Request-scoped
Skill content	In-process	Application lifetime
MCP tool results	Redis	5 minutes
User context	Redis	15 minutes

13. Error Handling

13.1 Exception Hierarchy

```
class MAFGAError(Exception):
    """Base exception for MAFGA system."""
    pass

class AgentError(MAFGAError):
    """Agent execution errors."""
    pass

class OrchestrationError(MAFGAError):
    """Orchestration errors."""
    pass
```

```
class MCPError(MAFGAError):
    """MCP protocol errors."""
    pass

class AuthenticationError(MAFGAError):
    """Authentication/authorization errors."""
    pass
```

13.2 Error Response Format

```
{
  "status": "error",
  "error": {
    "code": "AGENT_ERROR",
    "message": "Failed to invoke agent",
    "details": {
      "agent_name": "MerchPlanner",
      "original_error": "LLM timeout"
    }
  },
  "request_id": "req-abc123"
}
```

14. Testing Strategy

14.1 Test Pyramid

E2E Tests (API tests)	5%
Integration Tests	25%
Unit Tests	70%

14.2 Test Organization

```
tests/
├── unit/
│   ├── test_agents.py
│   ├── test_orchestration.py
│   ├── test_mcp.py
│   └── test_middleware.py
├── integration/
│   ├── test_api_agents.py
│   ├── test_api_orchestration.py
│   └── test_redis_memory.py
```

```
└─ e2e/
   └─ test_full_workflow.py
   └─ test_hitl_approval.py
```

14.3 Mock Strategy

```
@pytest.fixture
def mock_chat_client():
    """Mock LLM client for testing."""
    client = AsyncMock()
    client.chat_completion = AsyncMock(return_value={
        "content": "Mock response",
        "tool_calls": None,
        "model": "mock-model",
    })
    return client

@pytest.fixture
def mock_agent(mock_chat_client):
    """Mock agent for testing orchestration."""
    agent = MagicMock()
    agent.name = "MockAgent"
    agent.invoke = AsyncMock(return_value=AgentResponse(
        content="Agent result",
        agent_name="MockAgent",
    ))
    return agent
```